



MACHINE LEARNING FOR GRAPHS

E(3)-EQUIVARIANT VARIATIONAL FLOW MATCHING FOR MOLECULAR GENERATION

MATÚŠ HALÁK

STUDENTNUMBER: 2724858

Abstract. We present *ABC-Transformer*, an E(3)-equivariant graph transformer for joint generation of discrete molecular graphs (**A**toms, **B**onds), together with continuous 3D atomic positions (**C**oordinates). Our E(3)-equivariant graph transformer builds on the graph transformer introduced in DiGress. For generative training and sampling, we utilize the *variational flow matching* (VFM) framework. We reproduce the original VFM results on discrete molecular generation with the original architecture. When trained on joint molecular generation, our ABC-VFM achieves performance competitive with other recent methods.

Keywords: flow matching · equivariance · graph transformer · molecules

1 Introduction

Generative modeling studies how to produce novel realistic samples from intractable target distributions. A useful approach, used by many state-of-the-art generative models, is to learn a mapping from some initial distribution to the intractable target distribution. Learning such mappings enables generation of realistic samples by sampling from known initial distributions. Methods using this perspective include diffusion[1], normalizing flows[8], and flow matching [10].

One of most promising and impactful application domains for generative modeling is molecular design. The astronomical number of possible molecule configurations makes expert-guided search through this space a bottleneck for discovery of novel drugs and materials. Generative modeling promises to severely constrain this search space, by cheaply generating high-quality subsets of possible novel drugs or materials, easy to evaluate by human experts.

Graphs are a natural choice of representation for molecules, since nodes and edges of a graph can succinctly represent molecular atoms and bonds respectively. The graph representation also enables researchers to leverage modern graph machine learning tools, such as Graph Transformers [3] for molecular machine learning tasks, including molecular generation [16, 5].

Here, we first reproduce the results from the *Variational Flow Matching* (VFM) paper [5]. VFM introduces a generative objective and sampling framework, which is a novel perspective on flow matching, centered around parameterizing a variational approximation of the posterior distribution over sampling trajectory endpoints. This view is especially useful for categorical generative modeling and is thus naturally applicable to generating graphs with typed nodes and edges, like molecules. We move beyond categorical generative modeling and apply VFM to joint generative modeling of discrete molecular graphs together with continuous atomic coordinates. For this, we propose an $E(3)$ -equivariant extension of the graph transformer architecture, named **ABC-Transformer**. Evaluation of our architecture with VFM (ABC-VFM) on joint molecular generation shows results competitive with existing methods.

1.1 Contribution

Our key contributions¹ are

- **ABC Graph Transformer:** We successfully incorporate an $E(3)$ -equivariant attention-modulated coordinate update into the graph transformer [3, 16].
- **Results for Joint Molecular Generation:** We train and evaluate our ABC-Transformer with variational flow matching on joint molecular generation, achieving results competitive with recent approaches.

2 Related Work

Generative Modeling for Discrete Data. Several previous works explored generative modeling for discrete data. DiGress [16], introduced a discrete diffusion approach on top of a modified graph transformer architecture. We extend the DiGress architecture to joint modeling of discrete and continuous data, and use VFM instead of diffusion. The original VFM paper [5] used the DiGress graph transformer for discrete molecular generation. We apply VFM to joint molecular generation with our proposed ABC-Transformer architecture.

Group-Equivariant Generative Modeling for Continuous Data. Previous work on preserving geometrical symmetries in generative models for continuous data has used normalizing flows [15], flow matching [7], and variational flow matching [4] on top of "simple" $E(n)$ -equivariant GNN architectures (EGNN) [14]. We incorporate insights from EGNN into a Graph Transformer, applicable to joint generation of discrete and continuous data.

Joint Generative Modeling of Discrete and Continuous Data. Recently, multiple methods for joint generation of discrete and continuous data have surfaced. Two methods have used the SemlaFlow architecture [6] together with flow matching [6], or variational flow matching [4]. We introduce a new architecture for joint molecular generation, by adding an $E(3)$ -equivariant coordinate update to the Graph Transformer.

¹ All code and pre-trained model weights are available at github.com/matushalak/vfm.

3 Background

3.1 Continuous Normalizing Flows and Flow Matching

Continuous Normalizing Flows (CNFs) use a neural network v_t^θ to learn a time-dependent vector field $\frac{d}{dt}\varphi_t(x) = u_t(\varphi_t(x))$ representing a transformation φ which transforms samples from known initial distributions p_0 at $t = 0$ to samples from an intractable target distribution p_1 at $t = 1$.

Flow matching (FM) [10] is an efficient approach to learning CNFs, without performing ODE integration during training. FM uses the fact that, even without access to the true vector field $u_t(x)$, we can always construct per-example conditional trajectories $u_t(x|x_1)$ from any initial point x towards a specific target point x_1 . Probabilistically, u_t is then an expectation over the conditional trajectories weighted by endpoint posterior probabilities $p_t(x_1|x)$.

$$u_t(x) = \int u_t(x|x_1)p_t(x_1|x)dx_1 \quad p_t(x_1|x) = \frac{p_t(x|x_1)p_{data}(x_1)}{p_t(x)} \quad (1)$$

In FM, the conditional trajectories are defined as straight lines from x_t to x_1 .

$$u_t(x|x_1) = \frac{x_1 - x_t}{(1 - t)} \quad (2)$$

As a result, samples along the trajectories $x_t \sim p_t(x|x_1)$ can be obtained by linear interpolation between samples from a tractable prior $x_0 \sim p_0$ (e.g. $N(0, I)$) and target samples $x_1 \sim p_1$.

$$x_t = tx_1 + (1 - t)x_0 \quad (3)$$

FM learns to approximate $u_t(x)$ with a neural network $v_t^\theta(x)$, which can be efficiently trained on predicting the conditional trajectories (CFM).

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t \sim [0,1], x_1 \sim p_{data}(x_1), x_t \sim p_t(x|x_1)} [\|v_t^\theta(x_t) - u(x_t|x_1)\|_2^2] \quad (4)$$

For sampling, FM simply follows the predicted trajectory at each time-point:

$$x_{t+1} = x_t + v^\theta(x_t)\Delta t \quad x_{t=0} = x_0 \sim p_0 \quad (5)$$

3.2 Molecular Graphs and Molecular Generation Tasks

Discrete molecular generation, deals with generating *molecular graphs* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. $\mathcal{V}$ contains a one-hot matrix of the atom type of each node (atom) $\mathcal{V} \in \{0, 1\}^{n_{atoms} \times n_{atom_types}}$, and $\mathcal{E}$ contains a one-hot tensor of edge (bond) types between all pairs of nodes (atoms) in the graph (molecule), including an extra "no bond" category indicating absence of edge between two nodes $\mathcal{E} \in \{0, 1\}^{n_{atoms} \times n_{atoms} \times (n_{bond_types} + 1)}$. In discrete molecular generation, the 3D coordinates of all atoms $\mathcal{C} \in \mathbb{R}^{n_{atoms} \times 3}$ are either ignored, or inferred. **Continuous** molecular generation mainly deals with generating geometrically correct atomic coordinates $\mathcal{C}$, and infers atom types and bond types later based on the molecular geometry, for instance inferring bonds between atoms within a certain radius. **Joint** molecular generation tackles simultaneous generation of both chemically valid discrete molecular graphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, and geometrically consistent continuous atomic coordinates $\mathcal{C}$.

3.3 Group invariance and equivariance

A function $f(\cdot)$ is **invariant** to a group of transformations G , if $f(x) = f(g \cdot x)$ for all transformations $g \in G$. For instance, the sum function is invariant to any permutation P of the input elements x , since $\sum_i x_i = \sum_i (Px)_i$.

A function $f(\cdot)$ is **equivariant** to a group G , if $g \cdot f(x) = f(g \cdot x)$ for all transformations $g \in G$. For instance, self-attention (without positional embeddings) is permutation equivariant. Given a permutation matrix P , we can show that $P(QK^T V) = PQ(PK)^T PV$ (since $P^T P = I$).

4 Research Reproduction

We focus on reproducing the results of VFM [5] on discrete molecular generation, focusing on the QM9 benchmark [13]. Like the original paper, we use the graph transformer introduced in DiGress [16], as the backbone architecture.

4.1 Variational Flow Matching

VFM introduces a different perspective on flow matching, by approximating the posterior over flow endpoints $p_t(x_1|x)$ with a variational distribution $q_t^\theta(x_1|x)$. The key insight is that when we replace the intractable posterior from Equation 1 with the variational approximation, we learn a vector field $v_t^\theta(x)$, which approaches the optimal vector field $u_t(x)$ as we minimize the KL divergence between the exact and variational posteriors.

$$v_t^\theta(x) = \int u_t(x|x_1) q_t^\theta(x_1|x) dx_1, \quad v_t^\theta(x) = u_t(x) \text{ if } D_{KL}(p_t(x_1|x) || q_t^\theta(x_1|x)) = 0 \quad (6)$$

In their paper [5], the authors show that the VFM objective simplifies to

$$\mathcal{L}_{VFM}(\theta) = -\mathbb{E}_{t,x,x_1} [\log q_t^\theta(x_1|x)] \quad (7)$$

Mean-field Variational Flow Matching To make VFM tractable, the authors introduce Mean-Field VFM (MF-VFM) which hinges on two insights. *First*, because the conditional vector field $u_t(x|x_1)$ is linear, the expectation over this linear conditional vector field $\int u_t(x|x_1) p_t(x_1|x) dx_1$ is the same as the conditional vector field pointing towards the expectation of the posterior distribution $u_t(x|\mathbb{E}[p_t(x_1|x)])$. Therefore, as long as the expectations of the exact and variational posteriors match, $v_t^\theta = u_t$. *Second*, the consequence of only considering the expectations of the posterior distributions is that we can fully factorize the problem, since each dimension d of the expected value $\mathbb{E}_{p_t}[x_1|x]^d$ only depends on dimension d of the distribution $\mathbb{E}_{p_t}[x_1^d|x]$. This means v_t^θ approximates u_t as long as we can match individual dimensions of the expectations over the true and variational posteriors, without ever parameterizing the covariance of $p_t(x_1|x)$.

The factorization leads to the simplified objective

$$\mathcal{L}_{MF-VFM} = -\mathbb{E}_{t,x,x_1}[\sum_{d=1}^D \log q_t^\theta(x_1^d|x)], \quad \text{if } q_t^\theta(x_1|x) := \prod_{d=1}^D q_t^\theta(x_1^d|x) \quad (8)$$

Because VFM parametrizes the variational posterior over conditional trajectory endpoints $q_t^\theta(x_1|x)$ instead of the vector field v_t^θ , the vector field followed during sampling is computed from the expectation of the variational distribution. Practically, thanks to the outlined MF-VFM perspective, it is sufficient to view the output of our neural network as the expectation over the variational distribution $\mu_t^\theta(x) := \mathbb{E}_{q_t^\theta(x_1|x)}[x_1]$. Using Equation 2, this allows us to compute (Equation 9) and flow along (Equation 5) the conditional vector field:

$$v_t^\theta(x_t) = \frac{\mu_t^\theta(x_t) - x_t}{(1-t)} \quad (9)$$

CatFlow The MF-VFM approach is especially useful for categorical data, where the trajectory end-points live on a simplex. Using a neural network to approximate the expectation over the variational categorical distribution $\mu_\theta^{dk}(x_t) = q_t^\theta(x_1^d = k|x_t) = \mathbb{E}_{q_t^\theta(x_1|x_t)}[\mathbb{I}[x_1^d = k]]$, yields for each data dimension d a ‘probability’ that it belongs to category k . The advantage of this approach (CatFlow) is that we can ensure that our neural network $\mu_\theta^{dk}(x_t)$ outputs values that also live on the simplex, which ensures that all sampling trajectories (constructed according to Equation 9) flow towards a realistic part of the high-dimensional space. Finally, in case of CatFlow, the objective simplifies to a standard cross-entropy loss, which improves training speed, stability and convergence.

$$\mathcal{L}_{CatFlow}(\theta) = -\mathbb{E}_{t,x,x_1}[\sum_d^D \sum_k^K \mathbb{I}[x_1^d = k] \log \mu_\theta^{dk}(x_t)] \quad (10)$$

4.2 Experiments & Results

We evaluate our implementation of CatFlow on the QM9 dataset [13], focusing on molecular graphs and ignoring atomic coordinates. Using CatFlow, we train the DiGress transformer for 1000 epochs (see Appendix A for training setup details). The trained model is evaluated by assessing the quality of 10000 CatFlow-generated molecules. We compute validity (% of generated molecules without atomic valency violations, assessed by RDKit [9]), uniqueness (% of valid molecules that are unique), Frechet Chemnet Distance (FCD: distance between activations of a pre-trained ChemNet model [12] to generated molecules versus test-set molecules), and novelty (% of unique molecules not in the training set).

Table 1 shows the results of our CatFlow reproduction, compared to several relevant baselines from the VFM paper [5]. Our reproduction qualitatively reproduces the original results of CatFlow, despite achieving slightly worse performance than the original implementation on most metrics, with the exception of FCD where our implementation beats all baselines. Figure 1 shows molecular graph samples generated by our CatFlow model.

Table 1. Results for discrete molecular generation on QM9. All approaches use the DiGress transformer backbone architecture. All baseline results taken from [5].

Method	Valid $\uparrow$	Unique $\uparrow$	FCD $\downarrow$	Novel $\uparrow$
DiGress [16]	99.00	96.20	-	-
Flow Matching [10]	94.10	98.20	5.155	-
CatFlow [5]	99.81	99.95	0.441	49.00
CatFlow (ours)	99.12	95.89	0.338	40.68

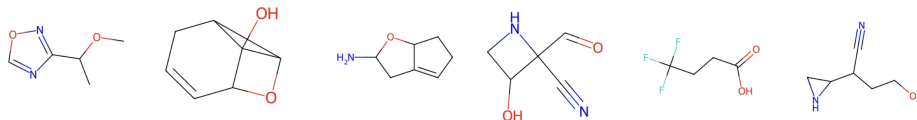


Fig. 1. Example QM9-like molecular graphs sampled by our CatFlow model.

5 Research Extension

Our main goal is to move from discrete to joint molecular generation, while ensuring that our network respects 3D symmetry of molecules. We design atomic coordinate updates equivariant to the transformation group $E(3)$ (rotations, reflections, translations). Our approach is inspired by simple equivariant updates introduced in EGNNs [14]. Here we incorporate similar updates into the graph transformer architecture. We provide an overview of the full proposed **ABC-Transformer** architecture in Appendix B. A comprehensive overview of the original graph transformer architecture can be found in [3, 16].

Given a molecular graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with N nodes (atoms) and corresponding atomic coordinates $\mathcal{C}$, we denote the single node’s features with $x_i \in \mathbb{R}^{d_x}$, single edge’s features $e_{ij} \in \mathbb{R}^{d_e}$, and a node’s 3D coordinates with $c_i \in \mathbb{R}^3$. We first compute pairwise distances between all atom coordinates $d_{ij} \in \mathbb{R}_+$, followed by unit-length pairwise relative coordinate vectors, $\hat{r}_{ij} \in \mathbb{R}^3$. Normalization ensures $\hat{r}_{ij}$ only provides information about potential direction of coordinate updates. Finally, we compute richer pairwise distance features $u_{ij} \in \mathbb{R}^{k'}$, by first expanding d_{ij} scalars over k radial basis functions (centered evenly between 0 – 11 Å, which represent extremes over possible pairwise atom distances in QM9), and then enriching the RBF representation with an MLP.

$$d_{ij} = \|c_i - c_j\|_2, \quad \hat{r}_{ij} = \frac{1}{d_{ij}}(c_i - c_j), \quad u_{ij} = \text{MLP}(\text{RBF}(d_{ij})) \quad (11)$$

We compute multi-headed scaled-dot-product attention scores $a_{ij}^{(h)} \in (0, 1)$ based on the queries and keys derived from the node features. Note that these attention scores differ from the scaled-outer-product attention scores $\alpha_{ij}^{(h)} \in (0, 1)^{d_f}$ used in the graph transformer to combine edge and node features.

$$a_{ij}^{(h)} = \text{softmax}\left(\frac{1}{\sqrt{d_f}} q_i^{(h)} k_j^{(h)T}\right), \quad q, k = \mathcal{V}W_q, \mathcal{V}W_k \in \mathbb{R}^{N \times (hd_f)} \quad (12)$$

Each head of our coordinate block learns a time-dependent learning rate $\eta_t^{(h)} \in \mathbb{R}_+$ using graph-level features y . Furthermore, each head also learns scalar gates $s_{ij}^{(h)} \in (-1, 1)$, using node, edge, coordinate and graph-level features.

$$\eta_t = \text{MLP}(y) \in \mathbb{R}_+^h, \quad s_{ij} = \text{MLP}(x_i || x_j || e_{ij} || u_{ij} || y) \in (-1, 1)^h \quad (13)$$

Theorem 1. Because all quantities derived from d_{ij} are E(3)-invariant, and $\hat{r}_{ij}$ is E(3)-equivariant, the following coordinate update is also E(3)-equivariant.

$$c_i^{t+1} = c_i^t + \Delta c_i, \quad \Delta c_i = \sum_{h=1}^H \eta_t^{(h)} \sum_{j=1}^N (a_{ij}^{(h)} s_{ij}^{(h)} \hat{r}_{ij}) \quad (14)$$

We prove Theorem 1 in Appendix C. Conceptually, a_{ij} decides whether to consider changing c_i towards c_j . s_{ij} determines magnitude and sign of the step, and $\hat{r}_{ij}$ decides direction of the step.

To train **ABC-VFM**, we introduce an E(3)-invariant loss on the expected conditional trajectory endpoints $c_t^\theta(c_1|c)$. Because Δc is E(3)-equivariant, the loss should not penalize flowing toward $c_t^\theta = g \cdot c_1$ if $g \in E(3)$. Thus, we use the MSE between E(3)-invariant pairwise distances derived from the predicted c_t^θ and the ground truth c_1 . Consequently, ABC-VFM learns to flow towards correct relative geometries, instead of arbitrary absolute positions in 3D space.

$$\mathcal{L}_{Coords}(\theta) = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N (\|c_i^\theta - c_j^\theta\|_2 - \|c_{1,i} - c_{1,j}\|_2)^2 \quad (15)$$

5.1 Experiments & Results

We evaluate our ABC-VFM approach on joint molecular generation by utilizing both molecular graphs and coordinates present from the QM9 dataset [13]. Following the same training and sampling procedure our the reproduction experiment (Appendix A), we evaluate ABC-VFM samples on discrete molecular graph metrics discussed above, as well as, two molecular geometry metrics. Due to poor documentation, we were unable to evaluate molecular geometry with the same metrics as [4], and instead devised the following metrics. 3D-embedding success, measures the % of valid molecules successfully embedded into a stable 3D-conformation using RDKit [9]. This metric is stricter than validity, because many ‘valid’ molecules might still result in physically unstable 3D-conformations. For successfully 3D-embedded molecules, we further use RDKit to find the optimal 3D-conformation. Because ABC-VFM is E(3)-equivariant, we use an E(3)-invariant evaluation metric, namely MAE between the pairwise atomic distances of the optimal 3D-conformation, and the generated 3D-conformation.

Table 2 shows the joint molecular generation results of our ABC-VFM model compared to relevant baselines from [4]. ABC-VFM achieves validity and uniqueness competitive with the baselines, and mostly improves the discrete molecular metrics relative to our CatFlow implementation. ABC-VFM also generates

promising molecular geometries (Figure 2), with MAE of 0.572 Å between pairwise atomic distances of "optimal" and generated molecules, which is around half the length of the shortest possible bond in the QM9 dataset.

Table 2. Results for joint molecular generation on QM9. All baseline results from [4].

Method	Valid $\uparrow$	Unique $\uparrow$	FCD $\downarrow$	Novel $\uparrow$	3D-Embed $\uparrow$	MAE $\downarrow$
SemlaFlow [6]	99.4	97.9	-	-	-	-
G-VFM [4]	98.9	97.5	-	-	-	-
ABC-VFM (ours)	99.01	96.24	0.324	41.05	96.73	0.572

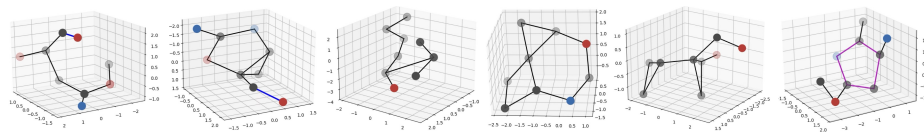


Fig. 2. Example QM9-like 3D molecules sampled by our ABC-VFM model.

6 Discussion & Conclusion

Here, we successfully reproduced the results of VFM on discrete molecular generation [5], and extended the graph transformer with an E(3)-equivariant update for node coordinates, achieving promising results on joint molecular generation. We attribute slight differences in performance between our CatFlow implementation and the original one, to slight differences in implementation. Additionally, some implementation details from the VFM paper did not match their code, complicating full reproducibility. Finally, we consider our extension successful, as ABC-VFM improved performance on discrete metrics compared to our CatFlow with a vanilla graph transformer, and overall showed competitive performance with recent joint molecular generative approaches [6, 4].

Our approach has several limitations. We faced the general problem finding useful molecular quality metrics. While most metrics were (implicitly) based on simple valency checking without considering more chemically-relevant (geometric) stability; other metrics were poorly documented in the literature, preventing their use. Next, in ABC-VFM discrete molecular features are not informed by molecular geometry, as we found this derailed training towards harmful local minima. Finally, in ABC-VFM, the graph-level features are severely underrepresented compared to DiGress. Nevertheless, our strong performance challenges the need to use more complex graph-level features in graph transformers.

Future directions. The most exciting extension of this work would be applying it to property-guided molecular generation. While a recent paper obtained excellent results using a simple EGNN-FM approach ([17]), using ABC-VFM instead could lead to further improvements. Finally, although MF-VFM recovers the optimal vector field under assumptions of linear trajectories, future research could investigate whether estimating the posterior covariance can be used for uncertainty estimation, or to dynamically adjust sampling step-size.

References

1. Albergo, M.S., Boffi, N.M., Vanden-Eijnden, E.: Stochastic interpolants: A unifying framework for flows and diffusions. arXiv preprint arXiv:2303.08797 (2023)
2. Corso, G. *et al.*: Principal Neighbourhood Aggregation for Graph Nets, en (2020). <https://doi.org/10.48550/arXiv.2004.05718>. <http://arxiv.org/abs/2004.05718> (visited on 01/12/2026). arXiv:2004.05718 [cs].
3. Dwivedi, V.P., Bresson, X.: A Generalization of Transformer Networks to Graphs, en (2021). <https://doi.org/10.48550/arXiv.2012.09699>. <http://arxiv.org/abs/2012.09699> (visited on 01/12/2026). arXiv:2012.09699 [cs].
4. Eijkelboom, F. *et al.*: Controlled Generation with Equivariant Variational Flow Matching, en (2025). <https://doi.org/10.48550/arXiv.2506.18340>. <http://arxiv.org/abs/2506.18340> (visited on 01/12/2026). arXiv:2506.18340 [cs].
5. Eijkelboom, F. *et al.*: Variational Flow Matching for Graph Generation, en (2024). <https://doi.org/10.48550/arXiv.2406.04843>. <http://arxiv.org/abs/2406.04843> (visited on 01/12/2026). arXiv:2406.04843 [cs].
6. Irwin, R. *et al.*: SemlaFlow – Efficient 3D Molecular Generation with Latent Attention and Equivariant Flow Matching, en (2025). <https://doi.org/10.48550/arXiv.2406.07266>. <http://arxiv.org/abs/2406.07266> (visited on 01/12/2026). arXiv:2406.07266 [cs].
7. Klein, L., Krämer, A., Noé, F.: Equivariant flow matching, en (2023). <https://doi.org/10.48550/arXiv.2306.15030>. <http://arxiv.org/abs/2306.15030> (visited on 01/12/2026). arXiv:2306.15030 [stat].
8. Kobayez, I., Prince, S.J.D., Brubaker, M.A.: Normalizing Flows: An Introduction and Review of Current Methods. IEEE Transactions on Pattern Analysis and Machine Intelligence **43**(11), 3964–3979 (2021). <https://doi.org/10.1109/TPAMI.2020.2992934>. <http://arxiv.org/abs/1908.09257> (visited on 01/12/2026)
9. Landrum, G.: RDKit: Open-source cheminformatics, <http://www.rdkit.org>
10. Lipman, Y. *et al.*: Flow Matching for Generative Modeling, (2023). arXiv: 2210.02747 [cs.LG]. <https://arxiv.org/abs/2210.02747>.
11. Perez, E. *et al.*: FiLM: Visual Reasoning with a General Conditioning Layer, en (2017). <https://doi.org/10.48550/arXiv.1709.07871>. <http://arxiv.org/abs/1709.07871> (visited on 01/12/2026). arXiv:1709.07871 [cs].
12. Preuer, K. *et al.*: Fréchet ChemNet Distance: A metric for generative models for molecules in drug discovery, (2018). arXiv: 1803.09518 [cs.LG]. <https://arxiv.org/abs/1803.09518>.
13. Ramakrishnan, R. *et al.*: Quantum chemistry structures and properties of 134 kilo molecules. Scientific data **1**(1), 1–7 (2014)
14. Satorras, V.G., Hoogeboom, E., Welling, M.: E(n) Equivariant Graph Neural Networks, en (2021). <https://doi.org/10.48550/arXiv.2102.09844>. <http://arxiv.org/abs/2102.09844> (visited on 01/12/2026). arXiv:2102.09844 [cs].
15. Satorras, V.G. *et al.*: E(n) Equivariant Normalizing Flows, en (2021). <https://doi.org/10.48550/arXiv.2105.09016>. <http://arxiv.org/abs/2105.09016> (visited on 01/12/2026). arXiv:2105.09016 [cs].
16. Vignac, C. *et al.*: DiGress: Discrete Denoising diffusion for graph generation, en (2023). <https://doi.org/10.48550/arXiv.2209.14734>. <http://arxiv.org/abs/2209.14734> (visited on 01/12/2026). arXiv:2209.14734 [cs].
17. Zeng, C. *et al.*: PropMolFlow: property-guided molecule generation with geometry-complete flow matching. Nature Computational Science (2026)

A Experimental setup details

To improve computational efficiency, all hydrogens are removed from the QM9 molecules. We handle variable molecule sizes via zero-padding to the maximum size within each batch, utilizing node-masking throughout all architecture stages. For evaluation, we sample 10,000 molecules by drawing 100 molecule sizes from the QM9 empirical distribution and sampling 100 molecules per size.

The training and sampling hyperparameters are summarized in Table 3.

Table 3. Training and sampling hyperparameters for both models.

Hyperparameter	ourCatFlow	ABC-VFM
Transformer layers		6
Training epochs		1000
Batch size		1024
Learning rate		1×10^{-3}
Optimizer	AdamW (weight decay 1×10^{-12})	
Scheduler	Cosine Annealing	
Sampling step size (Δt)		1×10^{-2}
Training time (H100)	~10h	~14h

Specific architectural dimensions (e.g., MLP widths) are documented in `config.py`.

B ABC-Transformer details

The **ABC**-Transformer extends the Graph Transformer architecture to handle four concurrent data streams: **A**tom (node) features (X), **B**ond (edge) features (E), **C**oordinates (C), and global features (y). The architecture consists of a lifting stage, a series of L transformer layers, and a projection stage. The node, edge, and global feature streams work exactly the same as in DiGress [16], and are therefore only briefly outlined. Our thoroughly commented implementation can be found in `architecture.py`

B.1 Four Feature Streams

The model maintains four feature streams throughout its depth. Below we explain how each stream is updated within the `E3NodeEdgeCoordBlock`:

- **Global features stream** (y): Unlike DiGress, which computes a number of structural and spectral global features for each graph, the only global feature we input to our transformer is the sampling time corresponding to each graph. After the first step however, the global features include summarized information about node and edge features as well. Concretely, global

features are updated by mixture of current global features and Principal Neighborhood Aggregation (PNA) [2] over node and edge features.

$$y^{t+1} = \text{MLP}(y^t) + \text{PNA}(X^t) + \text{PNA}(E^t) \quad (16)$$

The coordinate-based $\text{PNA}(C^t)$ was omitted in the final version of the y update because it destabilized training and led to bad local minima.

- **Node features stream** (X): Updated via multi-head scaled-outer-product-attention. Global features are incorporated using FiLM (Feature-Wise Linear Modulation) [11] layers:

$$X^{t+1} = \text{FiLM}(y, \text{Attention}(X, E)) \quad (17)$$

The attention mechanism incorporates edge features E directly into the attention scores before the softmax. We considered concatenating the enriched pairwise distance features U to the edge features E before their combination with the attention scores, but found that this destabilized training and led to bad local minima.

- **Edge features stream** (E): Edge features are updated by flattening the multi-head scaled-outer-product attention scores and applying FiLM modulation from the global features y .

$$E^{t+1} = \text{FiLM}(y, \text{flatten}(\text{AttnScores})) \quad (18)$$

- **Coordinate features stream** (C): This stream is updated using an $E(3)$ -equivariant update rule. The update Δc is modulated by a temporal learning rate η_t derived from y , scaled dot-product attention scores a_{ij} , and a gating scalar s_{ij} derived from the concatenation of all streams. A detailed description is presented in Section 5 which presents our main contribution.

B.2 Lifting and Projection

- **Lifting:** Discrete atom types (X), bond types (E), and global features (y) are lifted to (higher) hidden dimensions using MLPs. Coordinates (C) are not lifted to preserve equivariance.
- **Projection:** After the transformer layers, MLPs map the latent X , E , and y streams back to their original dimensions. Coordinates remain in $\mathbb{R}^3$ throughout the entire network.

B.3 Symmetry Preservation

The Graph Transformer architecture ensures permutation equivariance by using neighborhood aggregation and self-attention, as shown in [16]. $E(3)$ -equivariance of the coordinate stream is maintained by ensuring that all internal scalar features are derived from invariant distances d_{ij} , while the coordinate update only shifts c_i along the direction of the equivariant unit vectors $\hat{r}_{ij}$, as shown in Appendix C.

C Coordinate stream in ABC-Transformer is E(3)-equivariant

Theorem 1. Because all quantities derived from d_{ij} are E(3)-invariant, and $\hat{r}_{ij}$ is E(3)-equivariant, the following coordinate update is also E(3)-equivariant.

$$c_i^{t+1} = c_i^t + \Delta c_i, \quad \Delta c_i = \sum_{h=1}^H \eta_t^{(h)} \sum_{j=1}^N (a_{ij}^{(h)} s_{ij}^{(h)} \hat{r}_{ij})$$

Proof: We recall that the group $E(3)$ consists of translations $t \in \mathbb{R}^3$ and orthogonal transformations (rotations and reflections) $R \in O(3)$. Thus, if $g \in E(3)$ we can define $g \cdot c = Rc + t$.

Next, we can observe the influence of $g \in E(3)$ on d_{ij} :

$$\begin{aligned} d_{ij}(g \cdot c_i, g \cdot c_j) &= \|(Rc_i + t) - (Rc_j + t)\|_2 = \|R(c_i - c_j)\|_2 \\ &= \sqrt{(c_i - c_j)^T R^T R (c_i - c_j)} \\ &= \sqrt{(c_i - c_j)^T (c_i - c_j)} \quad \text{since } R^T R = I \\ d_{ij}(g \cdot c_i, g \cdot c_j) &= d_{ij}(c_i, c_j) \end{aligned}$$

and conclude that d_{ij} is E(3)-invariant.

Next we can apply g to $\hat{r}_{ij}$:

$$\begin{aligned} r_{ij}(g \cdot c_i, g \cdot c_j) &= (Rc_i + t) - (Rc_j + t) = R(c_i - c_j) = Rr_{ij}(c_i, c_j) \\ \hat{r}_{ij}(g \cdot c_i, g \cdot c_j) &= \frac{(Rc_i + t) - (Rc_j + t)}{\|(Rc_i + t) - (Rc_j + t)\|_2} = \frac{Rr_{ij}}{d_{ij}} = R\hat{r}_{ij}(c_i, c_j) \end{aligned}$$

and conclude that $\hat{r}_{ij}$ is equivariant under $O(3)$ and invariant under translations, which is a special case of $E(3)$ equivariance.

Next, we review the scalars introduced in our model. η is derived from y , which does not depend on c , and is therefore invariant to $g \cdot c$. The same is true for a , which is derived from x , which do not depend on c . u is a function of d , but since we have shown that $d(g \cdot c) = d(c)$, u will also have the same property $u(d(g \cdot c)) = u(d(c))$. Finally, the only coordinate-related input to s is u , which we have just shown is E(3)-invariant, and thus we can apply the same argument $s(x, e, u(d(g \cdot c)), y) = s(x, e, u(d(c)), y)$.

Putting everything together, we can show that

$$\begin{aligned} \Delta c_i(x, e, g \cdot c, y) &= \sum_{h=1}^H \eta_t^{(h)} \sum_{j=1}^N (a_{ij}^{(h)} s_{ij}^{(h)} R\hat{r}_{ij}) = R \sum_{h=1}^H \eta_t^{(h)} \sum_{j=1}^N (a_{ij}^{(h)} s_{ij}^{(h)} \hat{r}_{ij}) \\ \Delta c_i(x, e, g \cdot c, y) &= R\Delta c_i(x, e, c, y) \end{aligned}$$

Meaning that Δc is O(3)-equivariant and invariant to translations, just like $\hat{r}$.

Finally, we consider applying the Δc_i to obtain new coordinates c_i^{t+1}

$$\begin{aligned}
c_i^{t+1}(x^t, e^t, g \cdot c^t, y^t) &= Rc_i^t + t + \Delta c_i(x^t, e^t, g \cdot c^t, y^t) \\
&= Rc_i^t + t + R\Delta c_i(x^t, e^t, c^t, y^t) \\
&= R(c_i^t + \Delta c_i(x^t, e^t, c^t, y^t)) + t \\
c_i^{t+1}(x^t, e^t, g \cdot c^t, y^t) &= g \cdot c_i^{t+1}(x^t, e^t, c^t, y^t)
\end{aligned}$$

showing that our coordinate update is E(3)-equivariant, completing the proof.